

《神经网络与深度学习》

模型独立的学习方式

周晓飞

xiaofei-zhou.com

自动化学院

模型独立的学习方式

- ▶ 这些学习方式不限于具体的模型

- ▶ 前馈神经网络、循环神经网络还是其他模型

- ▶ 然而一种学习方式往往会对符合某种特性的模型更加青睐

- ▶ 集成学习往往和方差大的模型组合时效果显著。

内容

- ▶ 集成学习
- ▶ 协同学习
- ▶ 多任务学习
- ▶ 迁移学习
- ▶ 终身学习
- ▶ 元学习

1 集成学习

集成学习

▶ 三个臭皮匠赛过诸葛亮

定理 10.1: 对于 M 个不同的模型 $f_1(\mathbf{x}), \dots, f_M(\mathbf{x})$, 平均期望错误为 $\bar{\mathcal{R}}(f)$ 。基于简单投票机制的集成模型 $f^{(c)}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x})$, 其期望错误在 $\frac{1}{M} \bar{\mathcal{R}}(f)$ 和 $\bar{\mathcal{R}}(f)$ 之间。

▶ 为了增加模型之间的差异性，可以采用Bagging和Boosting两类方法

集成方式

► Bagging类（自举策略）

- **Bagging (Bootstrap Aggregating)** 是一个通过不同模型的训练数据集的独立性来提高不同模型之间的独立性。我们在原始训练集上进行有放回的随机采样，得到M 比较小的训练集并训练M 个模型，然后通过投票的方法进行模型集成。
- **随机森林 (Random Forest)** 是在Bagging的基础上再引入了随机特征，进一步提高每个基模型之间的独立性。在随机森林中，每个基模型都是一棵决策树。

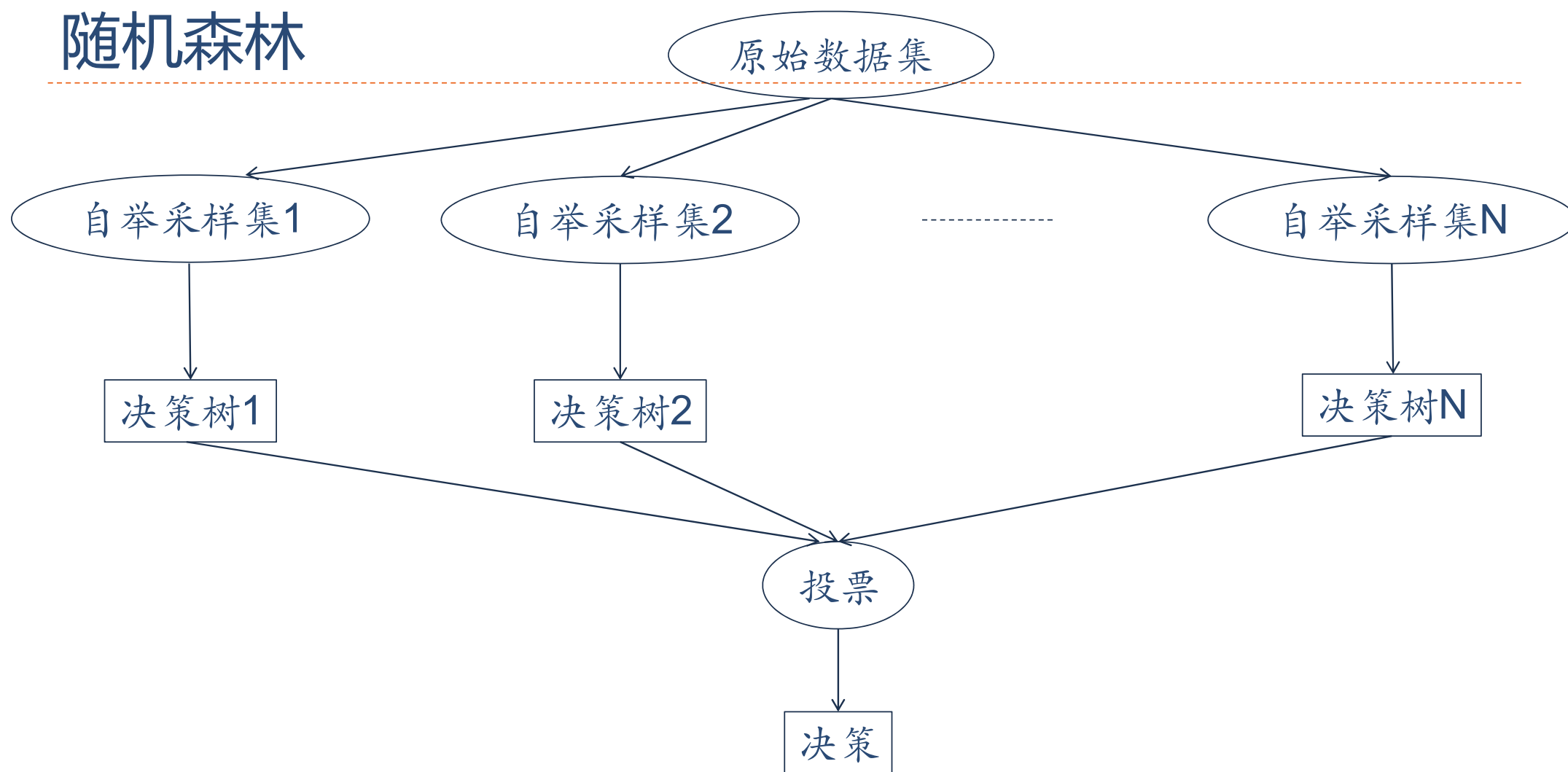
随机森林

► Bagging类（自举策略）

► 随机森林（Random Forest）

- ① 对样本数据进行自举重采样，得到多个样本集（有放回的随机抽取样本）
- ② 用每个重采样样本作为训练集，构造决策树（每次从候选特征中随机抽取 m 个特征）
- ③ 得到所需数目的决策树后，随机森林方法对这些树的输出进行投票，以票数最多的类作为随机森林的决策

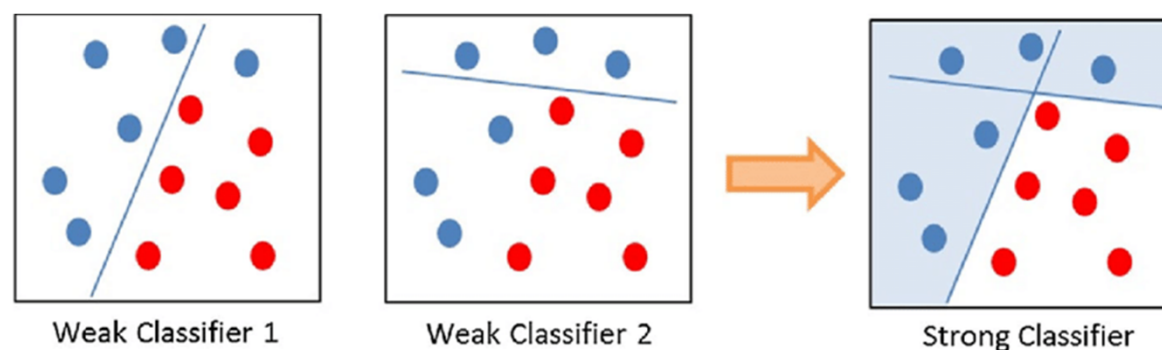
随机森林



集成方式

► Boosting类

- ▶ 按照一定的顺序来先后训练不同的基模型，每个模型都针对前序模型的错误进行专门训练。根据前序模型的结果，来调整训练训练样本的权重，从而增加不同基模型之间的差异性。
- ▶ AdaBoost (Adaptive Boosting) 算法



AdaBoost

Input :

- A training set $S = ((\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m))$.

Initialization :

- Maximum number of iterations T ;
- initialize the weight distribution $\forall i \in \{1, \dots, m\}, D^{(1)}(i) = \frac{1}{m}$.

for $t = 1, \dots, T$ **do**

- Learn a classifier $f_t : \mathbb{R}^d \rightarrow \{-1, +1\}$ using distribution $D^{(t)}$
- Set $\epsilon_t = \sum_{i: f_t(\mathbf{x}_i) \neq y_i} D^{(t)}(i)$
- Choose $a_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$
- Update the weight distribution over examples

$$\forall i \in \{1, \dots, m\}, D^{(t+1)}(i) = \frac{D^{(t)}(i) e^{-a_t y_i f_t(\mathbf{x}_i)}}{Z^{(t)}}$$

where $Z^{(t)} = \sum_{i=1}^m D^{(t)}(i) e^{-a_t y_i f_t(\mathbf{x}_i)}$ is a normalization factor such that $D^{(t+1)}$ remains a distribution.

Output : The voted classifier $\forall \mathbf{x}, F(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T a_t f_t(\mathbf{x}) \right)$

2 自训练和协同训练

自训练 (Self-Training or Self-Teaching)

▶ 自训练亦称作自举法，具体思路如下：

- ▶ 1) 使用标注数据来训练一个模型，
- ▶ 2) 使用这个模型来预测无标注样本的标签
- ▶ 3) 把预测置信度较高的样本及其预测的伪标签加入训练集
- ▶ 4) 重新训练新的模型，并不断重复这个过程

▶ 自训练缺点

- ▶ 无法保证每次加入训练集的样本的伪标签是正确的
- ▶ 关键在于如何设置挑选样本的标准

算法 10.2: 自训练的训练过程

输入: 标注数据集 $\mathcal{L} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$;

无标注数据集 $\mathcal{U} = \{\mathbf{x}^{(m)}\}_{m=1}^M$;

迭代次数 T ; 每次迭代增加样本数量 P ;

1 **for** $t = 1 \dots T$ **do**

2 根据训练集 $\mathcal{L}$, 训练模型 f ;

3 使用模型 f 对无标注数据集 $\mathcal{U}$ 的样本进行预测, 选出预测置信度
高的 P 个样本 $\mathcal{P} = \{(\mathbf{x}^{(p)}, f(\mathbf{x}^{(p)}))\}_{p=1}^P$;

4 更新训练集:

$$\mathcal{L} \leftarrow \mathcal{L} \cup \mathcal{P}, \quad \mathcal{U} \leftarrow \mathcal{U} - \mathcal{P}.$$

5 **end**

输出: 模型 f

协同训练 (Co-Training)

▶ 不同数据具有不同视角

- ▶ 条件独立 & 充足和冗余性

▶ Multi-View Learning

- ▶ 1) 根据不同视角分别训练两个模型 f_1 & f_2
- ▶ 2) 用 f_1 & f_2 在无标签数据集上进行预测
- ▶ 3) 各自 选取预测置信度高的样本加入训练集
- ▶ 4) 以此重新训练两个不同视角的模型，并不断重复

算法 10.3: 协同训练的训练过程

输入: 标注数据集 $\mathcal{L} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$;

无标注数据集 $\mathcal{U} = \{\mathbf{x}^{(m)}\}_{m=1}^M$;

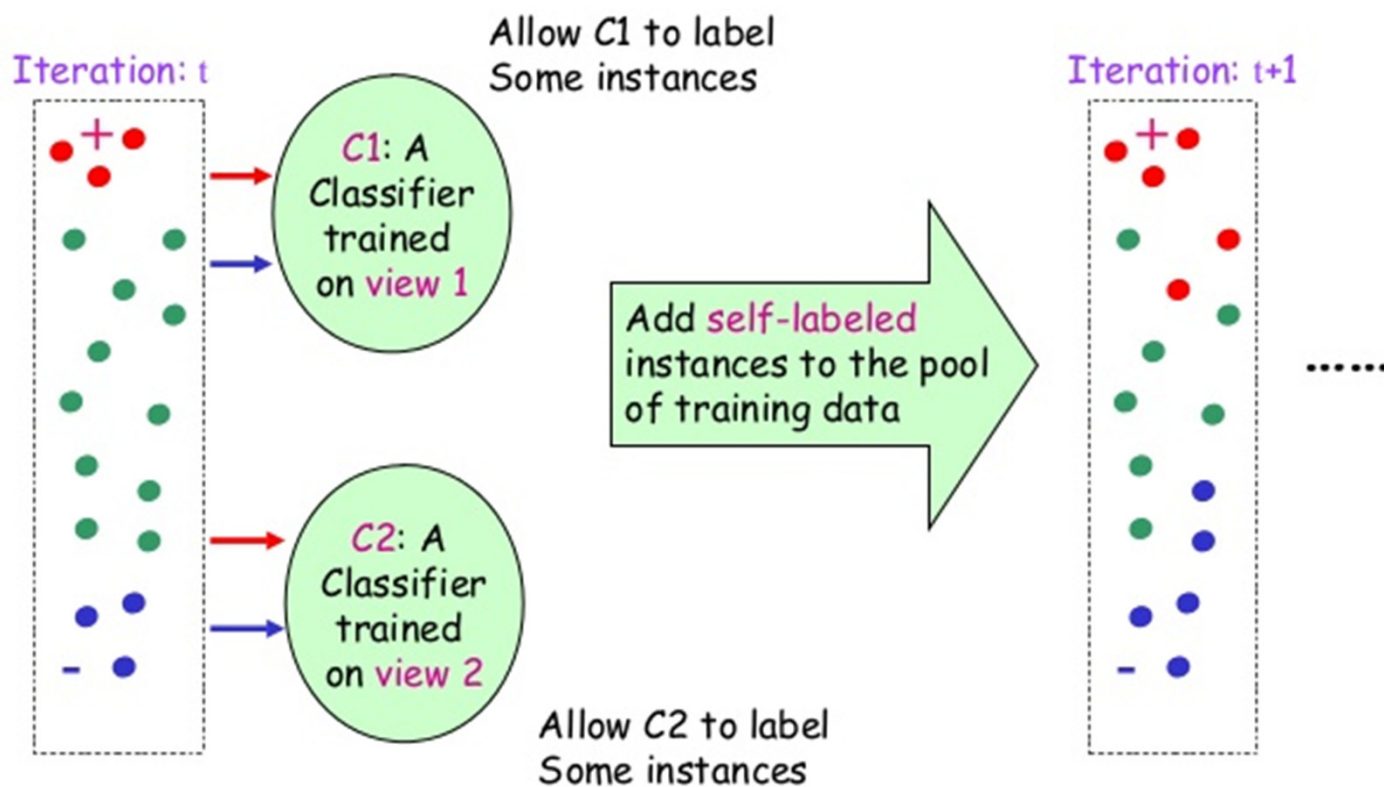
迭代次数 T ; 候选池大小 K ; 每次迭代增加样本数量 $2P$;

```
1 for  $t = 1 \cdots T$  do
2   根据训练集  $\mathcal{L}$  的视角  $V_1$  训练训练模型  $f_1$ ;
3   根据训练集  $\mathcal{L}$  的视角  $V_2$  训练训练模型  $f_2$ ;
4   从无标注数据集  $\mathcal{U}$  上随机选取一些样本放入候选池  $\mathcal{U}'$ , 使得
       $|\mathcal{U}'| = K$ ;
5   for  $f \in f_1, f_2$  do
6     使用模型  $f$  预测候选池  $\mathcal{U}'$  中的样本的伪标签;
7     for  $p = 1 \cdots P$  do
8       根据标签分布, 随机选取一个标签  $y$ ;
9       从  $\mathcal{U}'$  中选出伪标签为  $y$ , 并且预测置信度最高的样本  $\mathbf{x}$ ;
10      更新训练集:
          
$$\mathcal{L} \leftarrow \mathcal{L} \cup \{(\mathbf{x}, y)\}, \quad \mathcal{U}' \leftarrow \mathcal{U}' - \{(\mathbf{x}, y)\}.$$

11      end
12    end
13 end
```

输出: 模型 f_1, f_2

Co-Training



3 多任务学习

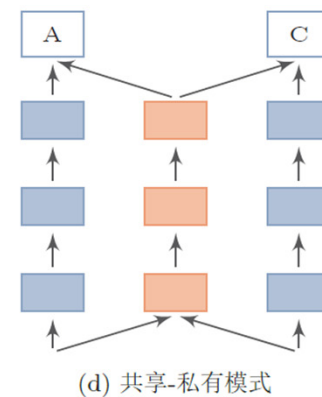
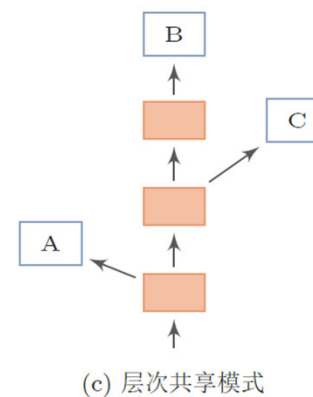
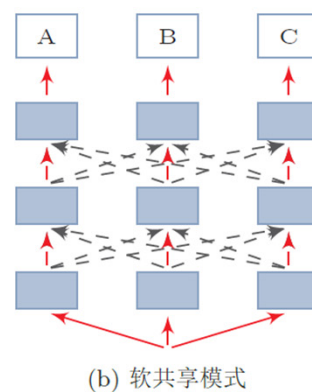
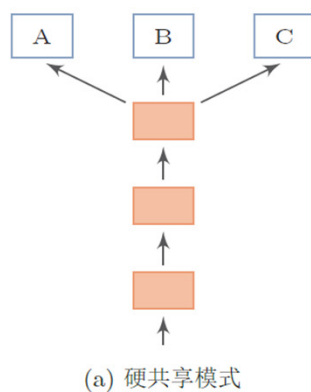
多任务学习 (Multitask Learning)

▶ 多任务学习

- ▶ 同时学习多个相关任务，实现知识共享，利用彼此之间的相关性来改进模型的性能和泛化能力
- ▶ 归纳迁移学习 (Inductive transfer learning)

▶ 共享机制

- ▶ 1) 硬共享模式
- ▶ 2) 软共享模式
- ▶ 3) 层次共享模式
- ▶ 4) 共享-私有模式



Multitask Learning*

Machine Learning 1997

RICH CARUANA

School of Computer Science, Carnegie Mellon University, Pittsburgh, PA 15213

caruana@cs.cmu.edu

Multitask Learning is an approach to **inductive transfer** that improves **generalization** by using the domain information contained in the training signals of related tasks as an **inductive bias**. It does this by learning tasks in parallel while using a **shared representation**; what is learned for each task can help other tasks be learned better.

多任务学习 (Multitask Learning)

► 学习步骤

► 交替训练

- M个相关任务，第m个任务的训练集为 D_m ，包含 N_m 个样本 $D_m = \left\{ \left(\mathbf{x}^{(m,n)}, y^{(m,n)} \right) \right\}_{n=1}^{N_m}$
- 对应的模型分别为 $f_m(\mathbf{x}; \theta)$ ，总的损失函数为

$$L(\theta) = \sum_{m=1}^M \sum_{n=1}^{N_m} \eta_m L_m \left(f_m \left(\mathbf{x}^{(m,n)}; \theta \right), y^{(m,n)} \right)$$

- L_m 表示第m个任务的损失函数， η_m 是第m个任务的权重

► 学习流程

- 1) 联合训练阶段：随机任务/随机小批量样本 来更新参数
- 2) 单任务精调阶段：每个任务在上一步得到参数的基础上精调 (Fine-tuning)

4 迁移学习

迁移学习 (Transfer Learning)

▶ 领域 Domain $D = (X, Y, p(x, y))$

- ▶ 两个领域，输入\输出空间或概率分布中至少有一个不同，则两个领域就被认为是不同的

▶ 迁移学习

- ▶ 两个不同领域的知识迁移过程，利用源领域 D_S 学习到的知识来帮助目标领域 D_T 上的学习任务
- ▶ 源域的训练样本数量一般远大于目标领域

▶ 迁移学习

- ▶ 归纳迁移学习 Inductive Transfer Learning, 模型迁移
- ▶ 转导迁移学习 Transductive Transfer Learning, 样本迁移

迁移学习 (Transfer Learning)

表 10.1 迁移学习和标准机器学习的比较

学习类型	样本空间	概率分布
标准机器学习	$\mathcal{X}_S = \mathcal{X}_T, \mathcal{Y}_S = \mathcal{Y}_T$	$p_S(x, y) = p_T(x, y)$
迁移学习	$\mathcal{X}_S \neq \mathcal{X}_T$ 或 $\mathcal{Y}_S \neq \mathcal{Y}_T$ 或 $p_S(x, y) \neq p_T(x, y)$	

迁移学习 (Transfer Learning)

► 归纳迁移学习

- 源域和目标域相关，输入空间相同，输出空间相同/不相同，任务不同
- 源域 D_s 中有大量的训练样本

► 1) 源域仅有大量无标注数据

- 源任务转换为无监督学习任务，类似于自学习和半监督学习

► 2) 源域有大量的标注数据，

- 可直接将源域训练模型迁移目标域, VGG/ResNet
- 两种迁移方式:
 - 基于特征
 - 精调Fine-tuning

迁移学习 (Transfer Learning)

▶ 转导迁移学习

- ▶ 样本到样本的迁移，领域适应 (domain adaptation)
- ▶ 假设源域有大量的标注数据，而目标域没有（或只有少量）标注数据，但有大量无标注数据，目标域的数据在训练阶段可见

- ▶ S & T 样本空间相同，数据分布不同 $p_S(\mathbf{x}, y) \neq p_T(\mathbf{x}, y)$

- ✓ 1) 协变量偏移 $p_S(\mathbf{x}) \neq p_T(\mathbf{x}), p_S(y|\mathbf{x}) = p_T(y|\mathbf{x})$

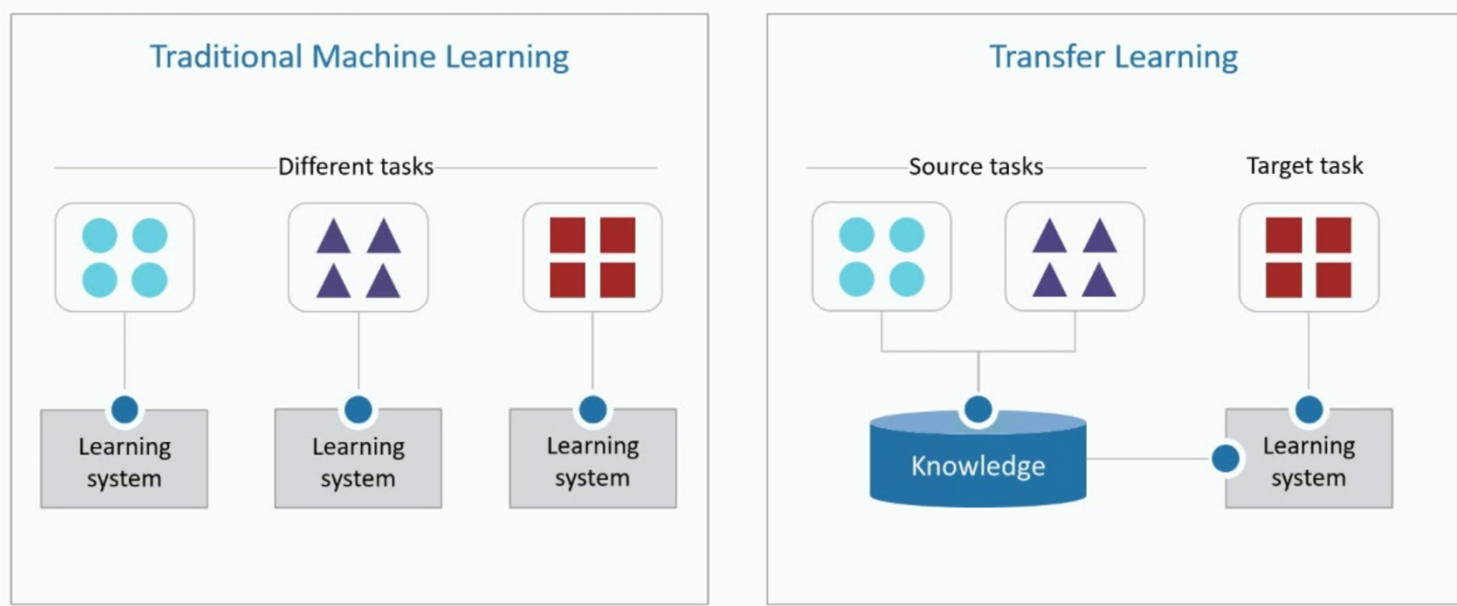
- ✓ 2) 概念偏移 $p_S(\mathbf{x}) = p_T(\mathbf{x}), p_S(y|\mathbf{x}) \neq p_T(y|\mathbf{x})$

- ✓ 3) 先验偏移 $p_S(y) \neq p_T(y), p_S(\mathbf{x}|y) = p_T(\mathbf{x}|y)$

$$p(\mathbf{x}, y) = p(\mathbf{x}|y)p(y) = p(y|\mathbf{x})p(\mathbf{x})$$

- ▶ 目前领域适应问题主要关注协变量偏移，关键在于如何学习领域无关的表示
 - ✓ MMD、CMD、对抗学习等

迁移学习 (Transfer Learning)





杨强，张宇，戴文渊，潘嘉林 著，
迁移学习，机械工业出版社，2020

迁移学习简明手册

一点心得体会

版本号：v1.1

王晋东

中国科学院计算技术研究所

tutorial.transferlearning.xyz

2018 年 4 月初稿

2019 年 10 月最新修改

王晋东，迁移学习简明手册，
<http://jd92.wang/>

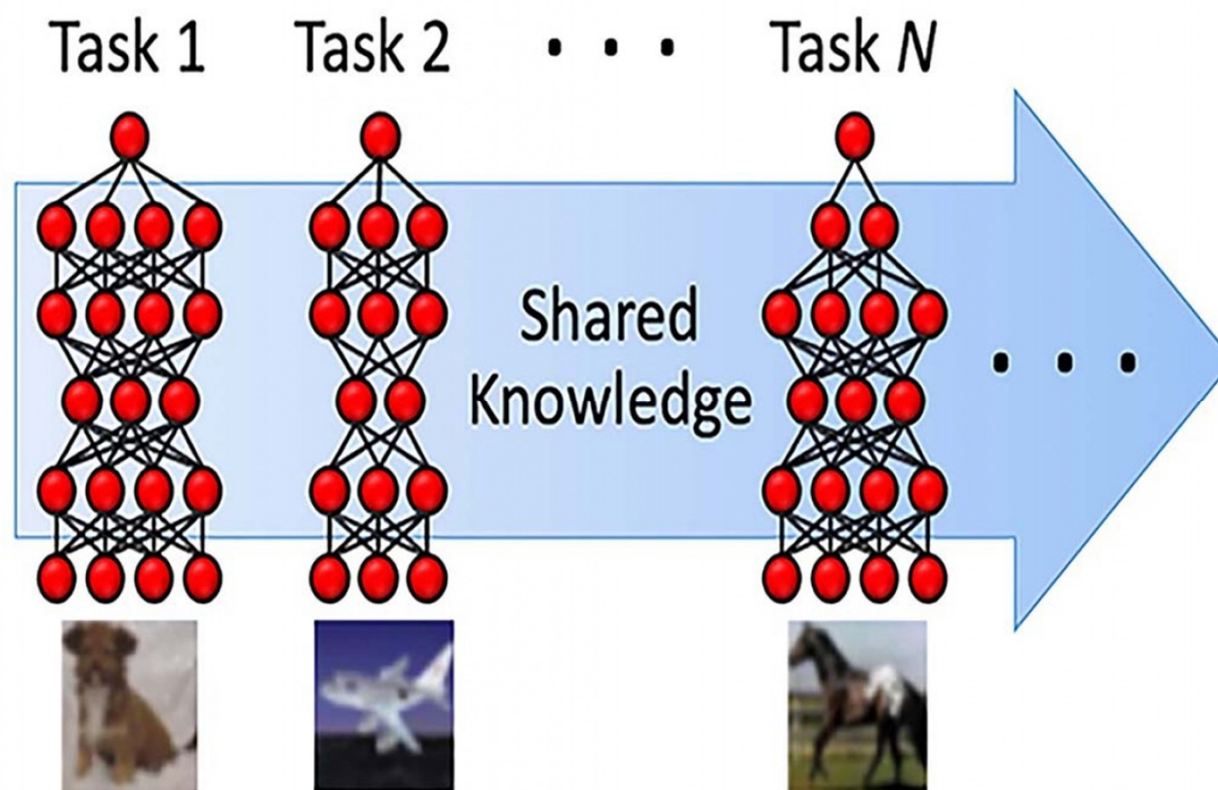
4 终身学习

终身学习 (Lifelong Learning)

▶ 终身学习

- ▶ 也叫持续学习 (continuous learning)
- ▶ 根据历史任务中学到的经验和知识，来帮助学习不断出现的新任务，并且这些经验和知识是持续累积的，不会因为新的任务而忘记旧的知识
- ▶ 假设一个终身学习算法已经在历史任务 $T_1, T_2, \dots, T_m$ 上学习到一个模型，当出现一个新的任务 T_{m+1} 时，这个算法可以根据过去在 **m个任务** 上学习的知识来帮助学习 **第m+1个任务**，同时累积 **所有的m+1个任务** 上的知识。
- ▶ 关键在于如何避免灾难性遗忘，即学习新任务的同时不忘记先前学会的历史任务
 - ✓ 弹性权重巩固

终身学习



6 元学习

元学习 (Meta Learning)

▶ 元学习

- ▶ 也称为学习的学习 (Learning to Learn)
- ▶ 动态的调整学习方式的能力
- ▶ 其目的是从已有任务中学习一种学习方法或元知识，可以加速新任务的学习，侧重从多种不同任务中归纳出一种学习方法

▶ 1) 基于优化器的元学习

- ▶ 目前神经网络的学习方法主要是定义一个目标损失函数 $L(\theta)$ ，并通过梯度下降算法来最小化 $L(\theta)$ ，并在不同的任务上选择不同的学习率和优化方法如动量法、Adam法。

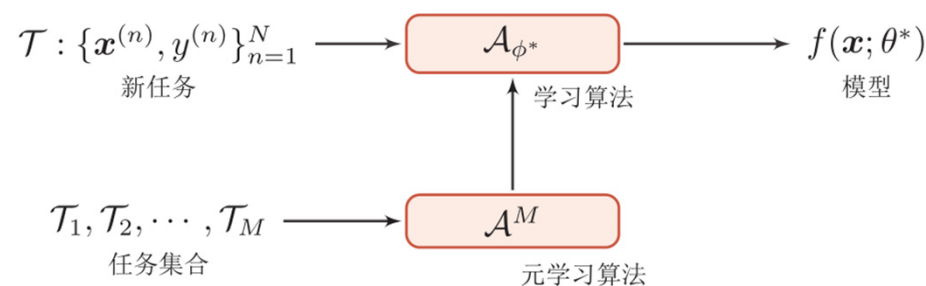
$$\theta_t \leftarrow \theta_{t-1} - \alpha \nabla L(\theta_{t-1})$$

元学习 (Meta Learning)

► 1) 基于优化器的元学习

- 不同的优化算法的区别在于更新参数的规则不同，因此一种很自然的元学习就是自动学习一种更新参数的规则，即通过另一个神经网络（比如循环神经网络）来建模梯度下降的过程。

- 基于优化器的元学习的示例



- 用 $g(\cdot)$ 来预测第 t 步时参数的更新差值 $\Delta\theta_t = \theta_t - \theta_{t-1}$ ，函数 $g(\cdot)$ 称为优化器，

$$\theta_{t+1} = \theta_t + g_t(\nabla L(\theta_t); \phi)$$

元学习 (Meta Learning)

► 1) 基于优化器的元学习

- ▶ 学习优化器 $\mathbf{g}_t(\cdot)$ 的过程可以看做是一种元学习过程，其目标是找到一个适用于多个不同任务的优化器。
- ▶ 在标准梯度下降中，每步迭代的目标是使得 $L(\theta)$ 下降。而在优化器的元学习中，我们希望在每步迭代的目标是 $L(\theta)$ 最小，具体的目标函数为

$$L(\phi) = E_f \left[\sum_{t=1}^T w_t L(\theta_t) \right]$$

$$\theta_t = \theta_{t-1} + \mathbf{g}_t(\cdot)$$

$$[\mathbf{g}_t; \mathbf{h}_t] = LSTM(\nabla L(\theta_{t-1}), \mathbf{h}_{t-1}; \phi)$$

其中 T 为最大迭代次数， $w_t > 0$ 为每一步的权重，一般可以设置 $w_t = 1, \forall t$

元学习 (Meta Learning)

► 2) 模型无关的元学习 (Model-Agnostic Meta-Learning, MAML)

- **思想**: 假设所有的任务都来自于一个任务空间, 其分布为 $p(T)$, 我们可以在这个任务空间的所有任务上学习一种**通用**的表示 f_θ , 这种表示可以经过梯度下降方法在一个特定的单任务 T_m 上进行**精调**, 即**任务适配**

$$\theta'_m \leftarrow \theta - \alpha \nabla_{\theta} L_{T_m}(f_{\theta})$$

- **目标**: 学习一个参数 θ 使得其经过一个梯度迭代就可以在新任务上达到最好的性能

$$\min_{\theta} \sum_{T_m \sim p(T)} L_{T_m}(f_{\theta'_m}) = \min_{\theta} \sum_{T_m \sim p(T)} \underbrace{L_{T_m}(f(\theta - \alpha \nabla_{\theta} L_{T_m}(f_{\theta})))}_{\theta'_m}$$

元学习 (Meta Learning)

► 2) 模型无关的元学习 (Model-Agnostic Meta-Learning, MAML)

- **参数更新**: 在所有任务上的元优化 (Meta-optimization) 也采用梯度下降来进行优化, 即

$$\begin{aligned}\theta &\leftarrow \theta - \beta \nabla_{\theta} \sum_{m=1}^M L_{T_m}(f_{\theta_m}) \\ &= \theta - \beta \nabla_{\theta} \sum_{m=1}^M L_{T_m}(f_{\theta_m}) \left(I - \alpha \nabla_{\theta}^2 L_{T_m}(f_{\theta_m}) \right)\end{aligned}$$

- 其中 β 为元学习率

元学习 (Meta Learning)

算法 10.5: 模型无关的元学习过程

输入: 任务分布 $p(\mathcal{T})$;

最大迭代次数 T , 学习率 α, β ;

1 随机初始化参数 θ ;

2 **for** $t = 1 \dots T$ **do**

3 根据 $p(\mathcal{T})$ 采样一个任务集合 $\{\mathcal{T}_m\}_{m=1}^M$ **for** $m = 1 \dots M$ **do**

4 计算 $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_m}(f_{\theta})$;

5 计算任务适配的参数: $\theta'_m \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_m}(f_{\theta})$;

6 **end**

7 更新参数: $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{m=1}^M \mathcal{L}_{\mathcal{T}_m}(f_{\theta'_m})$;

8 **end**

输出: 模型 f_{θ}

总结

▶ 1 集成学习

- ▶ 随机森林 & AdaBoost & Dropout

▶ 2 半监督学习

- ▶ 自训练 self-training & 协同训练 co-training

▶ 3 多任务学习

- ▶ 硬共享 & 软共享 & 层次共享 & 共享-私有模式
- ▶ 步骤：联合训练 → 单任务精调

▶ 4 迁移学习

- ▶ 归纳迁移学习 & 转导迁移学习

▶ 5 终身学习

▶ 6 元学习

- ▶ 基于优化器的元学习 & 模型无关的元学习

谢 谢!